COMPUTER ENGINEERING

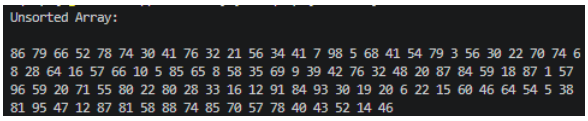
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	09/16/2026	**DATE SUBMITTED:**	09/16/2026
NAME:	Realubit, Seth Andrew I.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 8.1 Sorting Algorithms Pt2

1. PROCEDURE OUTPUT

What procedures did you perform?

Code + Console Screenshot	

sortingAlgo.h × main.cpp ×

HOA8p1 > PROCEDURES > main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include <algorithm>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArray(int arr[]);

int main(){
    int unsortedArr[maxSize];

    srand(time(0));

    for(int i=0; i < maxSize; i++){
        unsortedArr[i] = rand()%100;
    }

    std::cout<<"Unsorted Array: \n"<< std::endl;
    displayArray(unsortedArr);
    std::cout << std::endl;

    int shellSortCopy[maxSize];
    int mergeSortCopy[maxSize];
    int quickSortCopy[maxSize];

    std::copy(unsortedArr, unsortedArr + maxSize, shellSortCopy);
    std::copy(unsortedArr, unsortedArr + maxSize, mergeSortCopy);
    std::copy(unsortedArr, unsortedArr + maxSize, quickSortCopy);

    shellSort(shellSortCopy, maxSize);
    std::cout << "Shell Sort: \n" << std::endl;
    displayArray(shellSortCopy);


    mergeSort(mergeSortCopy, 0, maxSize - 1);
    std::cout << "\nMerge Sort: \n" << std::endl;
    displayArray(mergeSortCopy);



    quickSort(quickSortCopy, 0, maxSize - 1);
    std::cout << "\nQuick Sort: \n" << std::endl;
    displayArray(quickSortCopy);


    return 0;

}

void displayArray(int arr[]){
    for (int i = 0; i < maxSize; i++) {
    std::cout << arr[i] << " ";
}
    std::cout << std::endl;
}
```

Observations: In this code, I make use of the previous code from hands-on activity 7 and adding the algorithm library for making copies of the unsorted array so that the arrangement does not get messed up when shell sort already fixed the array and merge sort would try to fix that same array again.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot	

```
template <typename T>
void shellSort(T arr[], size_t arrSize) {
    for (size_t gap = arrSize / 2; gap > 0; gap /= 2) {
        for (size_t i = gap; i < arrSize; i++) {
            T temp = arr[i];
            int j;
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap) {
                arr[j] = arr[j - gap];
            }
            arr[j] = temp;
        }
    }
}
```

```
Shell Sort:

1 3 5 5 5 6 7 8 9 10 12 12 14 15 16 16 18 19 20 20 20 21 22 22 22 28 28 30 30 3
0 32 32 33 34 35 38 39 40 41 41 41 42 43 46 46 47 48 52 52 54 54 55 56 56 57 57
 57 58 58 59 59 60 64 64 65 66 66 68 68 69 70 70 71 74 74 74 76 76 78 78 79 79
80 80 81 81 84 84 85 85 86 87 87 87 88 91 93 95 96 98
```

Observations: The time complexity of shell sort is O(n log n) as its best case and the worst case as O(N^2) this is because it halves the array into half using the gap before sorting.

Table 8-2. Shell Sort Technique

Code + Console Screenshot	

```
Merge Sort:

1 3 5 5 5 6 7 8 9 10 12 12 14 15 16 16 18 19 20 20 20 21 22 22 22 28 28 30 30 3
0 32 32 33 34 35 38 39 40 41 41 41 42 43 46 46 47 48 52 52 54 54 55 56 56 57 57
 57 58 58 59 59 60 64 64 65 66 66 68 68 69 70 70 71 74 74 74 76 76 78 78 79 79
80 80 81 81 84 84 85 85 86 87 87 87 88 91 93 95 96 98
```

```cpp
template <typename T>
void merge(T arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    T* L = new T[n1];
    T* R = new T[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    delete[] L;
    delete[] R;
}

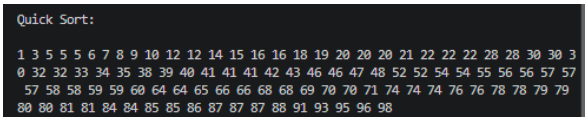
template <typename T>
void mergeSort(T arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}
```

Observations: For merge sort, we use the helper merge for combining what mergeSort halved because it is only responsible for getting the midpoint and splitting the range into half. Then merge is called for merge the already sorted from mergeSort.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	

```cpp
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]);
    return (i + 1);
}

template <typename T>
void quickSort(T arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

Observations: For quick sort, we have another helper which is the partition. This function picks a pivot and rearrange the array so that it follows the three conditions we have for quick sort.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

PROBLEM 1:

```cpp
template <typename T>
void insertionSortRange(T arr[], int low, int high){
    for (int K = low + 1; K <= high; K++){
        T temp = arr[K];
        int J = K - 1;

        while (J >= low && temp <= arr[J]){
            arr[J + 1] = arr[J];
            J--;
        }
        arr [J+1] = temp;
    }

}
```

Figure 1. Code Snippet of insertSortRange function

```
std::cout<<"Problem 1: "<<std::endl;
int pi = partition(testCopy, 0, maxSize - 1);
std::cout<<"Pivot placed on index: "<< pi <<std::endl;

insertionSortRange(testCopy, 0, pi - 1);
insertionSortRange(testCopy, pi + 1, maxSize -1);

displayArray(testCopy);
```

Figure 2. Code Snippet of main for problem 1

```
Problem 1:
Pivot placed on index: 90
1 2 2 2 3 3 3 6 6 8 8 8 10 10 11 16 17 18 21 21 21 22 22 26 26 29 29 29 30 31 31 31 3
2 32 33 33 33 37 37 38 38 39 41 42 43 43 44 44 45 47 47 48 48 49 49 49 51 51 53 53 54
 56 57 57 58 59 59 61 62 64 64 65 66 67 69 71 71 71 71 73 74 79 79 79 80 81 83 84 87
87 87 89 90 90 91 92 93 93 96 97
```

Figure 3. Snippet of the output in the terminal

Question: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

Answer: Yes, we can sort the left sub list and right sub list from the partition method in quick sort using other algorithms. In this case, I used insertion sort, and made another function inside the sortingAlgo.h followed with essentially the same logic inside the quick sort function. Then for the main we initialize the test/demo copy of the array and simply run it through the function. The first call of the insertionSortRange is for the left side then the second is for the right.

PROBLEM 2:

```
std::cout<<"\nProblem 2: "<<std::endl;
int prob2[15] = {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74};

std::vector<std::pair<std::string, std::function<void(int[], int)>>> algorithms = {
    {"Shell Sort", [](int arr[], int size){ shellSort(arr, size); }},
    {"Merge Sort", [](int arr[], int size){ mergeSort(arr, 0, size - 1); }},
    {"Quick Sort", [](int arr[], int size){ quickSort(arr, 0, size - 1); }}
};

for (const auto& [name, sortFunction]: algorithms){
    int copy[15];
    std::copy(prob2, prob2 + 15, copy);

    auto start = std::chrono::high_resolution_clock::now();
    sortFunction(copy, 15);
    auto end = std::chrono::high_resolution_clock::now();

    auto duration = std::chrono::duration_cast<std::chrono::nanoseconds>(end - start);
    std::cout << name << ": " << duration.count() << " ns" << std::endl;
}
```

Figure 1. Code Snippet of the main for problem 2

```
Problem 2:
Shell Sort: 900 ns
Merge Sort: 11300 ns
Quick Sort: 700 ns
```

Figure 2. Snippet of the output from the terminal

Question: What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

Answer: The sorting algorithm that gave me the fastest time performance proved to be quick sort as the fastest, followed by shell sort and merge sort. This came to be the case despite my initial assumption of shell sort being the fastest because of the small size of the array. Meanwhile, merge sort became the slowest because of how it worked, unlike the other two, merge sort splits the elements into individual pieces causing more memory, while the other two just sorts in place thus being faster. Merge sort and quick sort have a time complexity of $O(N \bullet log\ N)$ as its time complexity because of their process of divide-and-conquer that divides the array before sorting the data itself.

Conclusion:

In this hands-on activity we focused on the other part of sorting algorithms which are the divide-and-conquer sorting algorithms such as, shell sort, merge sort, and quick sort. Quick sort, is dividing the array by a pivot and basing it the operation based on that pivot. For Shell sort, it is an unstable version of insertion sort because of the gap that can prove to be unpredictable, the elements will be separated by the gap before getting sorted.

Then for merge sort, it cuts the array into individual pieces before merging while getting sorted.

In the supplementary activity, I mainly copied the quick sort logic and put it in the insertion sort and testing it, before that I initialized the algorithm library for its “std::copy” that allows me to make a copy of the “unsortedarr” so it essentially does not get touched in case I use it for different sorting algorithms. Then, for problem 2, I use new different libraries such as chrono, vector, and functional. Chrono is an advanced version of clock that gives me a more accurate reading of the time it took for getting the fastest sorting algorithm. Then, vector for containing the three different algorithms into a single variable that I can easily call rather than repeating the same code six times just for the three algorithms. Following that is the functional, this is essentially because merge, quick, and shell takes different things, and by using this we can use them without much further issue.

Overall, my understanding of the concept itself was decent enough to be able to code, but when the time complexity came, like in problem 2, I blanked out, because my initial solution would have been taking the amount of times they swapped before getting arranged which can be wrong at times and would might need a manual computation and visualization to get a proper answer.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of Concepts, Idea Generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I mainly used Claude AI this time for different possible solutions because unlike in problem 1 which was straight forward, I could not think of a possible solution for problem 2.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.’s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.